

Temel Kavramlar

Router, ağ üzerinde veri paketlerini hedefe doğru yönlendiren ara cihazdır. İnternette veri genellikle doğrudan değil, birden fazla router üzerinden iletilir.

Packet (paket), ağda taşınan veri birimidir ve iki bölümden oluşur: header ve payload. Header kısmında kaynak, hedef ve protokol bilgileri; payload kısmında ise asıl veri bulunur.

Protocol (protokol), cihazların birbiriyle hangi kurallara göre iletişim kuracağını belirler. Bu projede özellikle IP, UDP ve ICMP protokolleri önemlidir.

UDP, bağlantısız ve güvenilirlik garantisi vermeyen bir taşıma protokolüdür. Paketleri hızlı şekilde gönderir, ancak teslim garantisi sağlamaz. Bu nedenle traceroute uygulamalarında sıkça kullanılır.

Port, aynı cihaz üzerindeki farklı uygulamaları ayırt etmek için kullanılan mantıksal numaradır. UDP paketleri hedef sisteme ulaştığında hangi uygulamaya iletileceği port numarasına göre belirlenir.

IP (Internet Protocol), ağ üzerindeki cihazların adreslenmesini ve veri paketlerinin kaynaktan hedefe iletilmesini sağlayan temel ağ protokolüdür. Her cihazın kendine özgü bir IP adresi vardır ve paketler bu adresler üzerinden yönlendirilir. traceroute işleminde ulaşılmak istenen hedef makinenin IP adresini ifade eder.

sendsock, hedefe UDP paketleri göndermek için kullanılan sokettir.

recvsock, router'lerden veya hedef sistemden dönen ICMP mesajlarını almak için kullanılan sokettir.

TTL (Time to Live), bir paketin ağda geçebileceği en fazla hop(atlama noktası) sayısını gösterir. Her router bu değeri 1 azaltır. TTL sıfıra ulaştığında paket düşürülür ve kaynak cihaza hata mesajı gönderilir. Traceroute, bu mekanizmayı kullanarak yol üzerindeki router'ları sırasıyla keşfeder.

ICMP (Internet Control Message Protocol), ağ hata ve kontrol mesajları için kullanılan protokoldür. TTL süresi dolduğunda router, ICMP Time Exceeded mesajı gönderir. Paket hedefe ulaştığında ise hedef sistem genellikle ICMP Port Unreachable mesajı döndürür. Bu da hedefe ulaşıldığını gösterir.

Traceroute, hedefe artan TTL değerleriyle paket göndererek her adımda paketin hangi router'da sonlandığını belirleyen bir ağ tanılama yöntemidir. Böylece kaynak ile hedef arasındaki ara yönlendiriciler sıralı olarak tespit edilir.

Proje Özeti

Bu projenin temel amacı, Python programlama dili kullanılarak özgün bir traceroute uygulaması geliştirilmesidir. Proje yönergelerine göre uygulama Linux ve MacOS ortamlarında test edilmiştir. Windows kullanıcılarının ise uyumluluk sağlamak amacıyla Windows Subsystem for Linux (WSL) kurmaları gerekmektedir. Python tarafında önerilen sürüm Python 3.11'dir.

Kurulumun doğru biçimde tamamlandığını doğrulamak için, proje klasörü içerisinde `sudo python3 traceroute.py cmu.edu` komutunun çalıştırılması istenmektedir. Proje kapsamında öğrencinin yalnızca traceroute.py dosyasını düzenlemesi beklenmekte, diğer proje dosyalarında herhangi bir değişiklik yapılmaması özellikle belirtilmektedir.

Project 1A kapsamında öğrenciden, `traceroute(ip, sendsock, recvsock)` fonksiyonunu gerçekleştirmesi istenir. Bu fonksiyonun amacı, kaynak bilgisayar ile hedef makine arasında yer alan router'ları adım adım keşfetmektir. Fonksiyonun çıktısı, her hop mesafesi için bulunan router IP adreslerini içeren liste içinde listelerden oluşan bir veri yapısı olmalıdır. Başka bir ifadeyle, her iç liste belirli bir uzaklıktaki router veya router'ları temsil eder. Hedef IP adresine ulaşıldığında ise son eleman doğrudan `[ip]` biçiminde döndürülmelidir.

Proje 1: Kurulum

Bu projede, Python kullanarak temel bir `traceroute` uygulaması geliştirmeniz beklenmektedir. Gerekli temel bilgiler `Traceroute Guide` içerisinde verilmiştir.

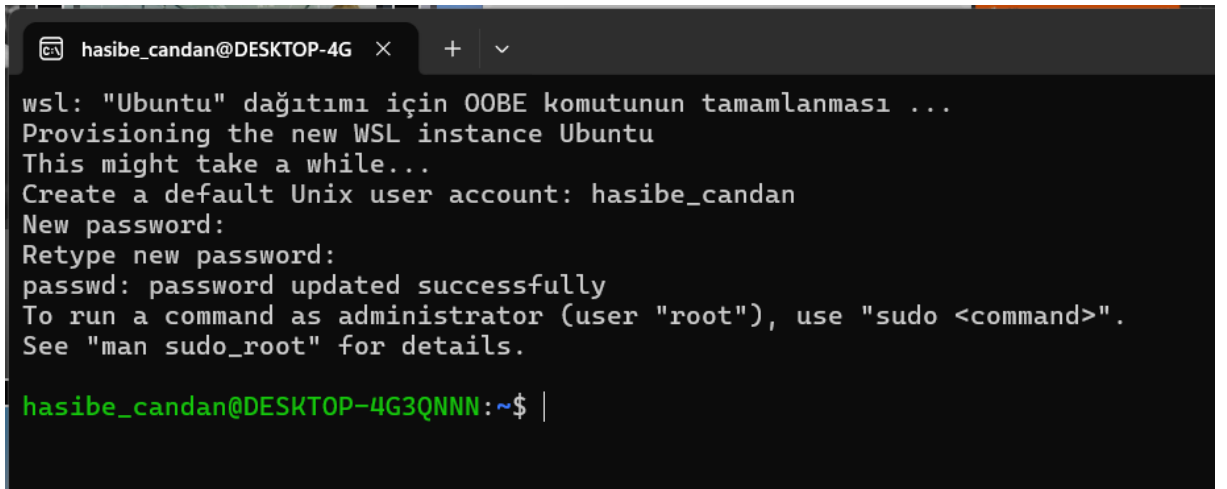
1. İşletim Sistemi Kurulumu

Bu proje, Linux ve macOS işletim sistemlerinde çalışacak şekilde test edilmiştir.

Windows kullanan öğrencilerin ise projeyi çalıştırabilmeleri için Windows Subsystem for Linux (WSL) kurmaları gerekmektedir. WSL, Windows üzerinde doğrudan bir Linux ortamı kullanmayı mümkün kılan bir araçtır.

WSL kurulumu için aşağıdaki adımlar izlenmelidir:

1. PowerShell uygulamasını Yönetici (Administrator) olarak açınız.
2. Aşağıdaki komutu çalıştırınız: `wsl --install`
3. Bu işlem sonucunda Ubuntu dağıtımı kurulacaktır. Sistem sizden yeniden başlatma isterse bilgisayarınızı yeniden başlatınız. Kontrol için `wsl --list --verbose`
4. Kurulum tamamlandıktan sonra Başlat menüsünden Ubuntu terminalini açınız.
5. Yine de Ubuntu açılmadı ve kurulmadıysa `wsl --install -d Ubuntu` bu komutu tekrar çalıştırınız.
6. İlk açılışta sizden bir kullanıcı adı ve şifre belirlemeniz istenecektir. İstedığınız bilgileri giriniz ve bunları not ediniz. Kurulum başarılıysa terminalde aşağıdakine benzer bir ifade görünmelidir: `username@hostname:~$`

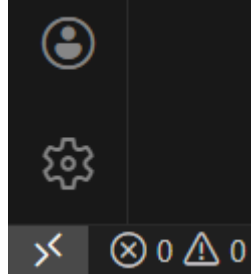


```
wsl: "Ubuntu" dağıtımı için OOB komutunun tamamlanması ...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: hasibe_candan
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hasibe_candan@DESKTOP-4G3QNNN:~$
```

Şekil1. Ubuntu

VS Code kullanan öğrenciler, sol alt köşedeki >< simgesine tıklayarak “Connect to WSL” seçeneği üzerinden Linux ortamına bağlanabilirler. Benzer bağlantı seçenekleri farklı kod editörlerinde de bulunabilir.



Şekil 2. VS Code’un sol alt kısmındaki işaret

2. Dosya Konumu ile İlgili Önemli Not

Windows kullanıcılarının dikkat etmesi gereken önemli bir husus vardır:

Başlangıç kodunu indirdikten sonra, Windows dosya sistemi altında bulunan klasörlerde (örneğin /mnt/c altında) Linux komutlarını çalıştırmamız önerilmez. Bunun nedeni, bu yönteminin çoğu durumda çok yavaş çalışması ve çalışma sürecini gereksiz biçimde verimsiz hâle getirmesidir.

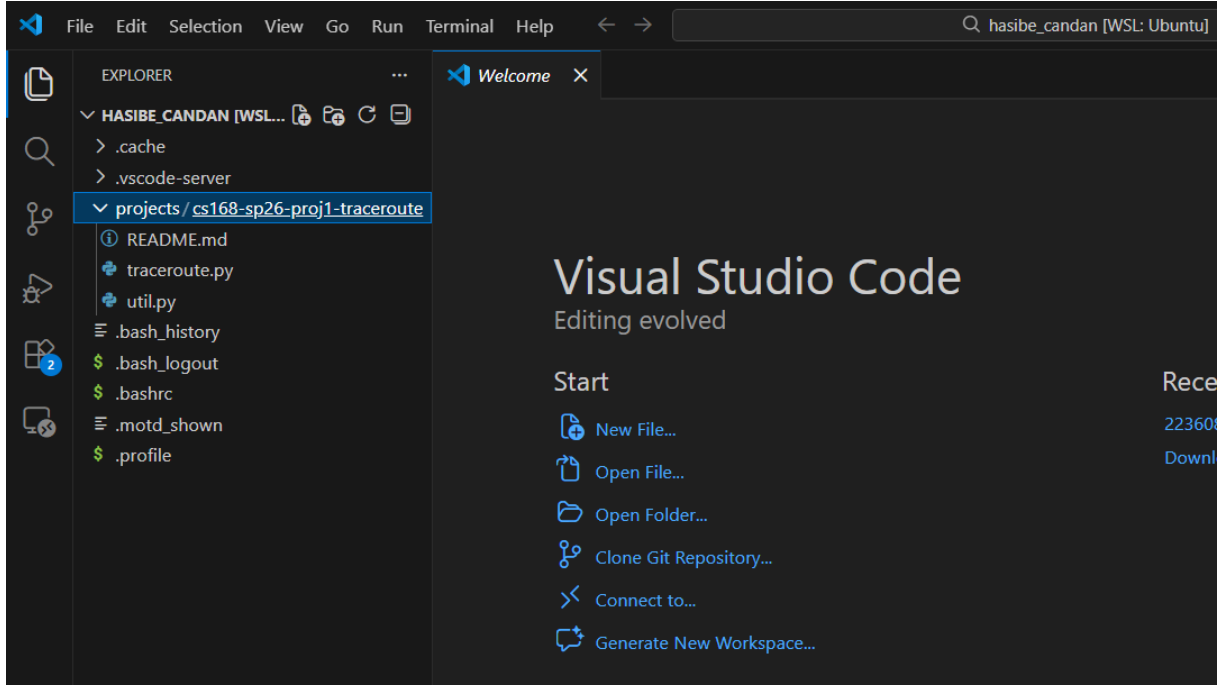
Bu nedenle, proje dosyalarını WSL içindeki Linux ev dizininizde tutmanız tavsiye edilir.

Öncelikle aşağıdaki komut ile bir proje klasörü oluşturunuz:

```
mkdir -p ~/projects
```

```
hasibe_candan@DESKTOP-4G3QNNN:~$ mkdir -p ~/projects
hasibe_candan@DESKTOP-4G3QNNN:~$ pwd
/home/hasibe_candan
hasibe_candan@DESKTOP-4G3QNNN:~$ ls
projects
hasibe_candan@DESKTOP-4G3QNNN:~$ |
```

Şekil 3. Ubuntu üzerinde klasör oluşturma



Şekil 6. VS Code üzerinden projeye erişim

3. Python Kurulumu

Bu proje Python 3.11 sürümü üzerinde test edilmiştir. Daha yeni Python sürümlerinde de çalışması mümkün olabilir; ancak bu durum garanti edilmemektedir.

Python sürümünüzü kontrol etmek için terminalde aşağıdaki komutlardan birini çalıştırınız:

`python3 --version` veya `python --version`

Eğer çıktı aşağıdaki biçimde görünüyorsa sisteminiz uygundur:

Python 3.11.*

4. Başlangıç Kodunun Kullanımı

Başlangıç kodunun bir kopyasını ilgili bağlantı üzerinden indiriniz.

Terminalde proje klasörüne geçmek için aşağıdaki komutu kullanınız:

`cd cs168-sp26-proj1-traceroute`

Bu projede yer alan tüm Python komutlarının bu dizin içerisinde çalıştırılması gerekmektedir.

5. Kurulumun Doğru Çalıştığının Kontrolü

Kurulumun başarılı olup olmadığını test etmek için terminalde aşağıdaki komutu çalıştırınız:

`sudo python3 traceroute.py cmu.edu`

Eğer aşağıdakine benzer bir çıktı elde ediyorsanız, kurulum büyük ölçüde doğru yapılmış demektir:

`traceroute to cmu.edu (128.2.42.10)`

1: *

2: *

3: *

(some lines omitted...)

26: *

27: *

28: *

29: *

30: *

6. Düzenlenecek Dosya

Bu proje kapsamında yalnızca traceroute.py dosyası düzenlenecektir.

Kod içerisinde, tamamlamanız gereken bölümler yorum satırları ile açık biçimde belirtilmiştir. Bu nedenle, dosyayı dikkatle incelemeniz ve yalnızca belirtilen alanlarda değişiklik yapmanız beklenmektedir.

7. Uygulama Kuralları

Projeyi tamamlarken aşağıdaki kurallara uyunuz:

- Başka dosyaları değiştirmeyiniz.
- Yeni dosya eklemeyiniz.
- Yeni import ifadesi eklemeyiniz.
- Yalnızca yorumlarda açıkça belirtilen bölümleri düzenleyiniz.
- Sabit değer içeren global değişkenler eklemeyiniz.
- Gerekli görülmesi hâlinde yardımcı fonksiyonlar ekleyebilirsiniz.

Proje 1A: Temel Traceroute

1. Projenin Genel Yapısı

Bu çalışma iki bölümden oluşmaktadır:

- **Proje 1A**
- **Proje 1B**

Proje 1B'ye geçmeden önce Proje 1A'nın tamamlanması gerekmektedir. Bu derste sadece 1A tamamlanacaktır. Proje 1A, öğrenciyi yönlendirmek amacıyla aşamalara ayrılmıştır. Bu aşamaların birebir sırayla izlenmesi zorunlu değildir; isteyen öğrenciler traceroute uygulamasını doğrudan sıfırdan da geliştirebilir. Ancak aşamalı yapı, kavramların sistematik biçimde öğrenilmesi açısından faydalıdır.

2. Test ve Hata Ayıklama

Otomatik deęerlendirici testleri yerel ortamda öęrenciye açık deęildir. Buna karřılık, öęrenciler çözümlerini Gradescope üzerinden istedikleri kadar gönderebilir. Gradescope üzerinde görölen puan, bu proje için nihai puan olarak kabul edilmektedir; ayrıca gizli test bulunmamaktadır.

Hata ayıklama için en yararlı yöntemlerden biri, yazdığınız traceroute uygulamasını gerçek bir hedef üzerinde çalıştırmak ve elde ettiğiniz sonucun anlamlı olup olmadığını incelemektir. Örnek komut aşağıdaki gibidir:

```
sudo python3 traceroute.py cmu.edu
```

```
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
22: *
23: *
24: *
25: *
26: *
27: *
28: *
29: *
30: *
```

Şekil 7. sudo python3 traceroute.py cmu.edu

3. Kodun Genel Yapısı

Bu projede temel olarak traceroute.py dosyası içerisindeki traceroute fonksiyonunu gerçekleştireceksiniz. Paket gönderme ve alma işlemleriyle ilgili bazı yardımcı işlevler ise util.py dosyasında hazır olarak verilmiştir. Bu dosya incelenebilir; ancak deęiřtirilmemelidir.

4. Görev: traceroute Fonksiyonunun Gerçekleştirilmesi

Amaç, sizin bilgisayarınız ile belirtilen hedef arasında bulunan tüm yönlendiricileri ortaya çıkaracak şekilde traceroute fonksiyonunu yazmaktır. Bu fonksiyon üç argüman alır: hedef adresi belirten ip, paket göndermek için kullanılacak sendsock, ve paket almak için kullanılacak recvsock.

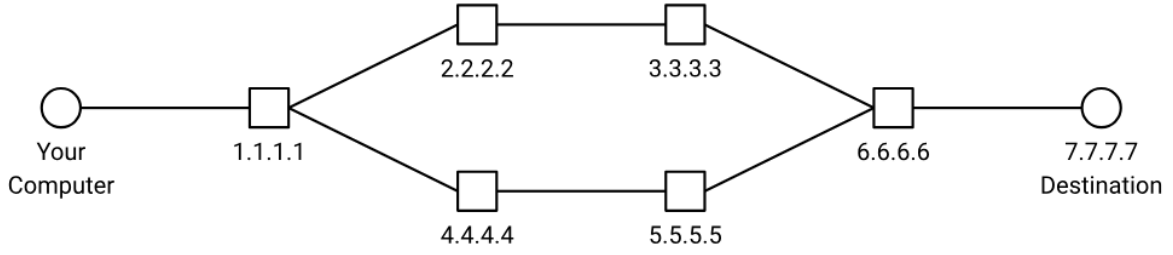
Fonksiyonun çıktısı listelerin listesi biçiminde olmalıdır. Beklenen yapı şu şekildedir:

- i indisli alt liste, i+1 uzaklıktaki yönlendiricilerin IP adreslerini içerir.
- Örneğin 0. alt liste, 1 hop uzaklıktaki yönlendiricileri gösterir.
- Her alt listedeki IP adresleri herhangi bir sırada olabilir.
- Belirli bir uzaklıkta yönlendirici bulunamazsa ilgili alt liste boş olabilir.
- Eğer hedef IP keşfedilirse, son alt liste yalnızca [ip] biçiminde olmalıdır.

Ayrıca, her TTL düzeyi için bulunan yönlendiriciler util.print_result ile ekrana yazdırılmalıdır. Yazdırılan çıktı notlandırılmasa da hata ayıklama açısından yararlıdır.

5. Beklenen Çıktıya Örnek

Varsayalım ki ağ topolojisi şöyle görünüyor,



Şekil 8. Ağ Topolojisi

ve iki farklı yol vardır ve hedef IP adresi 7.7.7.7 olsun. Böyle bir durumda, hata oluşmadığı ve gönderilen probe paketleri tüm yönlendiricilere ulaştığı varsayımı altında, traceroute fonksiyonunun aşağıdakine benzer bir yapı döndürmesi beklenir:

```
[["1.1.1.1"],  
["2.2.2.2", "4.4.4.4"],  
["3.3.3.3", "5.5.5.5"],  
["6.6.6.6"],  
["7.7.7.7"]]
```

Ayrıca ekrana da yaklaşık olarak şu biçimde bir çıktı yazdırılmalıdır:

traceroute to 7.7.7.7

```
1: 1.1.1.1  
2: 2.2.2.2  
   4.4.4.4  
3: 3.3.3.3  
   5.5.5.5  
4: 6.6.6.6  
5: 7.7.7.7
```

6. Paket Gönderme

sendsock nesnesi giden paketleri göndermek için kullanılır. Bu nesne üzerinde iki temel işlem öne çıkar:

- **sendsock.set_ttl(ttl)**: Sonraki paketler için TTL değerini ayarlar.
- **sendsock.sendto(msg.encode(), (ip, port))**: Belirtilen hedef IP ve porta bir UDP paketi gönderir. Burada msg paketin yükünü ifade eder ve encode() ile ham bayta dönüştürülür.

Örnek kullanım:

```
sendsock.set_ttl(12)
```



```
sendsock.sendto("Hello".encode(), ("4.4.4.4", 33434))
```

```
sendsock.set_ttl(20)
```

```
sendsock.sendto("Potato".encode(), ("5.5.5.5", 33464))
```

7. Paket Alma

recvsock nesnesi gelen paketleri almak için kullanılır. Temel yöntemler şunlardır:

- **recvsock.recv_select()**: Alınabilecek bir paket olup olmadığını kontrol eder. Eğer en az bir paket varsa True, zaman aşımı olursa False döndürür.
- **recvsock.recvfrom()**: Tek bir gelen paketi alır ve hem ham baytları hem de gönderen adres bilgisini döndürür.

Burada dikkat edilmesi gereken önemli nokta şudur: **recvfrom() çağrısından önce mutlaka recv_select() ile kontrol yapılmalıdır**; aksi hâlde alınacak paket yoksa hata oluşabilir.

Örnek kullanım:

```
if recvsock.recv_select():
```

```
    buf, address = recvsock.recvfrom()
```

```
    print(f"Packet bytes: {buf.hex()}")
```

```
    print(f"Packet is from IP: {address[0]}")
```

```
    print(f"Packet is from port: {address[1]}")
```

8. Sonucun Yazdırılması

`util.print_result(routers: list[str], ttl: int)` fonksiyonu, belirli bir TTL için bulunan yönlendiricileri düzenli biçimde ekrana yazdırmak amacıyla kullanılır. Bu fonksiyon, her TTL değeri için bir kez çağrılmalıdır.

Örnek kullanım:

```
util.print_result(["128.2.255.210", "128.2.42.10"], 7)
```

Bu çağrı, TTL değerini, IP adreslerini ve uygun durumlarda ilgili ad çözümlemelerini düzenli biçimde ekrana yazdırır.

9. Şimdi: Traceroute'u Manuel Olarak Çalıştırma

Bu aşamada dikkat edilmesi gereken pratik durum, gerçek ağ üzerindeki yönlendiriciler beklenmedik davranışlar gösterebilir. Bazıları hiç yanıt vermeyebilir, bazıları ise paket çözücülerin “geçersiz/bozuk paket” olarak yorumlayabileceği sıra dışı başlık alanları gönderebilir. Sayfada özellikle bazı gerçek yönlendiricilerin IPv4 Total Length alanını farklı biçimlerde doldurabildiği belirtilmektedir. Ancak otomatik değerlendircide kullanılan yönlendiriciler, yönergelerde tarif edilen biçimde davranacağından, bu sıra dışı gerçek dünya örneklerini kodunuzda ayrıca ele almanız gerekmemektedir.

Bu aşamada amaç, traceroute mantığını elle gözlemlemektir. Bu bölüm için ayrıca bir teslim beklenmemektedir; ancak sonraki aşamaları anlamak açısından yararlı görülmektedir. Öncelikle başlangıç kodundaki bazı satırlar geçici olarak yorum satırına alınır;

```

90 def traceroute(sendsock: util.Socket, recvsock: util.Socket, ip: str) \
91     -> list[list[str]]:
92
93     # TODO Add your implementation
94     ''' for ttl in range(1, TRACEROUTE_MAX_TTL+1):
95         util.print_result([], ttl)
96     return []
97     '''

```

Şekil 9. Yorum satırına al

Ardından TTL değeri 1 olan bir paket gönderilip gelen yanıtın ham hex baytları incelenir. Paket yükü kısa bir metin olabilir; örneğin "Potato". Kullanılacak hedef port numaraları başlangıç kodunda verilmiştir.

```

90 def traceroute(sendsock: util.Socket, recvsock: util.Socket, ip: str) \
91     -> list[list[str]]:
92     ttl = 1
93     sendsock.set_ttl(ttl)
94     sendsock.sendto("Potato".encode(), (ip, TRACEROUTE_PORT_NUMBER))
95
96     if recvsock.recv_select():
97         buf, address = recvsock.recvfrom()
98         print(f"Packet bytes: {buf.hex()}")
99         print(f"Packet is from IP: {address[0]}")
100        print(f"Packet is from port: {address[1]}")
101
102    return []

```

Şekil 10. İlgili değişiklik kodun içinde

Daha sonra aşağıdaki komutla kod çalıştırılır: `sudo python3 traceroute.py cmu.edu`

```

hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
28: *
29: *
30: *
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
traceroute to cmu.edu (128.2.42.10)
Packet bytes: 4500003e0004000008001dd93ac118001ac1185030b00d0400000000045000022e9f840000111b4b1ac11850380022a0aa9fc829a000ebed9
506f7461746f
Packet is from IP: 172.17.128.1
Packet is from port: 0
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$

```

Şekil 11. `sudo python3 traceroute.py cmu.edu`

Gelen yanıt paketinin baytları bir çevrimiçi paket çözücüye yapıştırılarak incelenebilir. Gerekirse paket çözücüye bunun bir IPv4 paketi olduğu belirtilmelidir; çünkü Layer 2 başlığı daha önce çıkarılmış durumdadır. Bu inceleme sırasında şu sorulara odaklanılması önerilir, bunları kısaca düşününüz ve araştırınız.

- Yanıt gerçekten kim gönderdi? Hedef makine mi, ara bir yönlendirici mi?
- Ara bir yönlendirici keşfedildi mi? Eğer keşfedildiyse IP adresi nasıl anlaşıldı?
- Yanıt paketi ne anlatmaya çalışıyor? Bir hata bildirimi mi yapıyor?

- Paket içinde hangi başlıklar var ve neden bulunuyorlar?

Sonrasında TTL değeri 2, 3, 4 veya 5 yapılarak süreç tekrar edilir. Bazı yönlendiriciler yanıt vermeyebileceği için birkaç farklı TTL denemek gerekebilir. Son olarak TTL değeri 30 yapılarak hedefe kadar ulaşıp ulaşılmadığı gözlenir.

```

90 def traceroute(sendsock: util.Socket, recvsock: util.Socket, ip: str) \
91     -> list[list[str]]:
92     ttl = 2
93     sendsock.set_ttl(ttl)
94     sendsock.sendto("Potato".encode(), (ip, TRACEROUTE_PORT_NUMBER))
95
96     if recvsock.recv_select():
97         buf, address = recvsock.recvfrom()
98         print(f"Packet bytes: {buf.hex()}")
99         print(f"Packet is from IP: {address[0]}")
100        print(f"Packet is from port: {address[1]}")
101
102    return []

```

Şekil 12. Ttl 2

```

hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
506f7461746f
Packet is from IP: 172.17.128.1
Packet is from port: 0
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
traceroute to cmu.edu (128.2.42.10)
Packet bytes: 45c0003e225300003f0165eeca0a80101ac1185030b0099560000000045000022024d400001119c5dac11850380022a0a8bf6829a000e13ca
506f7461746f
Packet is from IP: 192.168.1.1
Packet is from port: 0
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$

```

Şekil 13. ttl 2 olduğunda ekran çıktısı

TTL = 1 iken gelen yanıt 172.17.128.1 adresinden geldi. Bu, paketin ilk ara cihaza ulaşmış ve TTL değerinin burada bittiğini gösterir.

Paketteki ICMP Type 11, Code 0 bilgisi bunun bir “Time Exceeded” yanıtı olduğunu gösterir.

TTL = 2 iken yanıt 192.168.1.1 adresinden geldi.

Bu kez paket ilk hop’u geçmiş, ikinci hop’a ulaşmış ve TTL burada tükenmiştir.

Yani traceroute ikinci yönlendiriciyi de başarıyla keşfetmiştir.

IP adreslerinin değişmesi hata değil, tam tersine traceroute’un doğru çalıştığını gösterir.

TTL arttıkça paket daha uzak yönlendiricilere ulaşır.

Portun 0 görünmesi normaldir.

Çünkü gelen yanıt bir UDP uygulama cevabı değil, bir ICMP hata mesajıdır.

Hex verinin sonunda görülen `506f7461746f` ifadesi ASCII olarak “Potato” demektir.

Bu da yönlendiricinin, hangi pakete cevap verdiğini göstermek için senin gönderdiğin orijinal probe’un bir kısmını geri yolladığını gösterir.

Bu iki deneme, traceroute mantığının beklendiği gibi çalıştığını göstermektedir:

* TTL=1 → 1. hop keşfedildi

* TTL=2 → 2. hop keşfedildi

Hex Packet Decoder 9,423,682 packets decoded

172.17.128.1 → 172.17.133.3 ICMP Time-to-live exceeded (Time to live exceeded in transit)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
45	00	00	3E	00	04	00	00	80	01	DD	93	AC	11	80	01
AC	11	85	03	0B	00	D0	40	00	00	00	00	45	00	00	22
E9	F8	40	00	01	11	B4	B1	AC	11	85	03	80	02	2A	0A
A9	FC	82	9A	00	0E	BE	D9	50	6F	74	61	74	6F		

4 protocols in packet (76 bytes):

IPv4 ICMP IPv4 UDP

Şekil14. Hex Decoder1(<https://hpd.gasmi.net/>)

Hex Packet Decoder 9,423,679 packets decoded

192.168.1.1 → 172.17.133.3 ICMP Time-to-live exceeded (Time to live exceeded in transit)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
45	C0	00	3E	22	53	00	00	3F	01	65	EE	C0	A8	01	01
AC	11	85	03	0B	00	99	56	00	00	00	00	45	00	00	22
02	4D	40	00	01	11	9C	5D	AC	11	85	03	80	02	2A	0A
8B	F6	82	9A	00	0E	13	CA	50	6F	74	61	74	6F		

4 protocols in packet (76 bytes):

IPv4 ICMP IPv4 UDP

Şekil15. Hex Decoder2

Şekil 14 ve 15’de kısaca olay şudur. Bu şekilde görülen paket, aslında bir yönlendiricinin sana gönderdiği hata mesajıdır. Sen hedefe bir veri paketi yolluyorsun; ancak paket hedefe ulaşmadan yolda bir cihazda duruyor çünkü TTL değeri bitiyor. Bunun üzerine o cihaz sana “paketin buraya kadar geldi ama daha ileri gidemedi” anlamına gelen bir geri bildirim gönderiyor. Bu geri bildirimin içinde hem yönlendiricinin kendi hata mesajı, hem de senin gönderdiğin orijinal paketin bir kopyası bulunuyor. Böylece traceroute, paketin hangi cihaza kadar gidebildiğini anlayabiliyor.

Kısacası bu yapı, “paket burada durdu ve sebebi bu” (time-exceed) diyen bir ağ geri dönüş mesajıdır.

```
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
506f7461746f
Packet is from IP: 192.168.1.1
Packet is from port: 0
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$ sudo python3 traceroute.py cmu.edu
[sudo] password for hasibe_candan:
traceroute to cmu.edu (128.2.42.10)
Packet bytes: 45c00038039d0000fb01467651d4f208ac1185030b0087e0000000004500002238e74000011165c3ac11850380022a0ad654829a000e1422
Packet is from IP: 81.212.242.8
Packet is from port: 0
hasibe_candan@DESKTOP-4G3QNNN:~/projects/cs168-sp26-proj1-traceroute$
```

Şekil 16. Ttl 5 yazılmıştır.

Şekil 16’da TTL değeri 5 olarak ayarlanmıştır. Bu durumda gönderilen probe paketi, önceki denemelere göre ağda daha fazla yönlendirici geçebilmiş ve yanıt 81.212.242.8 adresinden alınmıştır. Bu, paketin beşinci atlamaya kadar ilerlediğini ancak bu noktada TTL değerinin tükendiğini göstermektedir. Hex Packet Decoder çıktısında görülen ICMP Time-to-live exceeded ifadesi de bunu doğrulamaktadır. Kısacası bu sonuç, TTL değeri arttıkça traceroute’un daha uzak ara düğümleri keşfedebildiğini ve 81.212.242.8 adresinin bu yol üzerindeki daha ileri bir yönlendirici olduğunu göstermektedir.

Sorulara kısa cevaplar:

* Yanıtı kim gönderdi?

Yanıtı hedef makine değil, ara bir yönlendirici göndermiştir. Çünkü TTL değeri hedefe ulaşmadan önce yolda tükenmiştir.

* Ara yönlendirici keşfedildi mi? IP adresi nasıl anlaşıldı?

Evet, ara yönlendirici keşfedilmiştir. Bunun IP adresi, terminal çıktısındaki “Packet is from IP” alanından ve paket çözümünde görülen kaynak IP adresinden anlaşılmıştır.

* Yanıt paketi ne anlatıyor? Bir hata bildirimi mi yapıyor?

Evet. Bu paket, CMP Time Exceeded mesajıdır. Yani yönlendirici, paketin TTL değeri bittiği için daha ileriye iletilemediğini bildirmektedir.

* Paket içinde hangi başlıklar var ve neden bulunuyorlar?

Paket içinde dış IPv4 başlığı, ICMP hata mesajı, ardından senin gönderdiğin paketin bir kısmı olan orijinal IPv4 ve UDP başlıkları bulunur. Bunun nedeni, yönlendiricinin hangi pakete yanıt verdiğini gönderen tarafa göstermek istemesidir.